Practicum: Open-loop regeling van een aeropendulum

Leerdoelen

- ☐ Je kunt de dynamische respons van een simpel open-loop systeem experimenteel bepalen.
- ☐ Je kunt meetgegevens in Python inlezen, verwerken en visualiseren.
- ☐ Je kunt een eenvoudig fysisch model vergelijken met experimentele meetresultaten.

Veiligheid

1. Houd tijdens de metingen voldoende afstand van de draaiende propeller en raak deze nooit aan wanneer de voeding is ingeschakeld.
2. Controleer vóór iedere meting of het aeropendulum vrij kan bewegen en of zich geen objecten binnen het bewegingsbereik bevinden.
3. Twijfel je over een handeling of apparaat? Vraag dit altijd aan de practicumbegeleider.

Introductie

Regeltechniek speelt een belangrijke rol binnen de technische natuurkunde. Veel technische systemen, zoals drones, robots, satellieten, productieprocessen en autonome voertuigen, moeten automatisch bestuurd worden om een gewenste positie of toestand te bereiken. Voordat een regelsysteem ontworpen kan worden, is het belangrijk om eerst het gedrag van het systeem zelf goed te begrijpen. Dit gebeurt vaak door het systeem in open-loop te bestuderen.

Bij een open-loop systeem wordt een ingangssignaal aangeboden zonder dat de uitgang wordt teruggekoppeld naar de regeling. De besturing corrigeert dus niet voor verstoringen of afwijkingen. Hoewel deze manier van regelen relatief eenvoudig is, vormt zij de basis voor vrijwel alle moderne regelsystemen. Door de respons van een systeem op verschillende ingangssignalen te meten, kunnen eigenschappen zoals de gevoeligheid, tijdconstante, demping en niet-lineariteiten worden bepaald.

Een eenvoudig voorbeeld van open-loop regeling is een magnetron. Je stelt bijvoorbeeld een vermogen van 800 W en een tijd van 2 minuten in. De magnetron voert deze instelling uit, maar meet niet of het eten daadwerkelijk de gewenste temperatuur heeft bereikt.

Is het eten na 2 minuten nog te koud? Dan corrigeert de magnetron dit niet automatisch. De uitgang (temperatuur van het eten) wordt namelijk niet teruggekoppeld naar de ingang (vermogen en verwarmingstijd).

In dit practicum onderzoek je het dynamische gedrag van een aeropendulum. Dit systeem bestaat uit een arm die om een draaipunt kan roteren en wordt aangedreven door een elektrische propeller. De propeller levert een stuwkracht die de arm omhoog beweegt, terwijl de zwaartekracht de arm terug naar beneden probeert te brengen. Door de motorspanning stapsgewijs te variëren ontstaat een evenwicht tussen deze twee krachten, waardoor verschillende stationaire hoeken en dynamische responsen kunnen worden bestudeerd.

Tijdens het practicum meet je de hoek van het aeropendulum als functie van de tijd voor verschillende motorspanningen. De meetgegevens worden vervolgens in geanalyseerd om de dynamische eigenschappen van het systeem te bepalen en te vergelijken met een eenvoudig fysisch model. De kennis die in dit practicum wordt opgedaan vormt de basis voor een vervolgpracticum, waarin hetzelfde systeem met behulp van terugkoppeling (*closed-loop control*) automatisch zal worden geregeld.

Theorie

Een aeropendulum bestaat uit een draaibare arm met aan het uiteinde een elektromotor en propeller. De propeller levert een stuwkracht waardoor de arm omhoog beweegt. De zwaartekracht werkt deze beweging tegen. Door het vermogen van de motor te variëren kan de positie van het aeropendulum worden beïnvloed.

In dit practicum wordt het systeem in **open-loop** onderzocht. Dit betekent dat het vermogen van de motor vooraf wordt ingesteld en niet automatisch wordt aangepast op basis van de gemeten hoek. Voor iedere instelling wordt vervolgens onderzocht hoe het aeropendulum reageert.

De propeller levert een stuwkracht F_T op een afstand l_T van het draaipunt. Het bijbehorende moment is

$$M_T = l_T F_T. \tag{1}$$

Tegelijkertijd oefent de zwaartekracht een moment uit op het systeem. Wanneer het zwaartepunt zich op een afstand l van het draaipunt bevindt, is dit moment

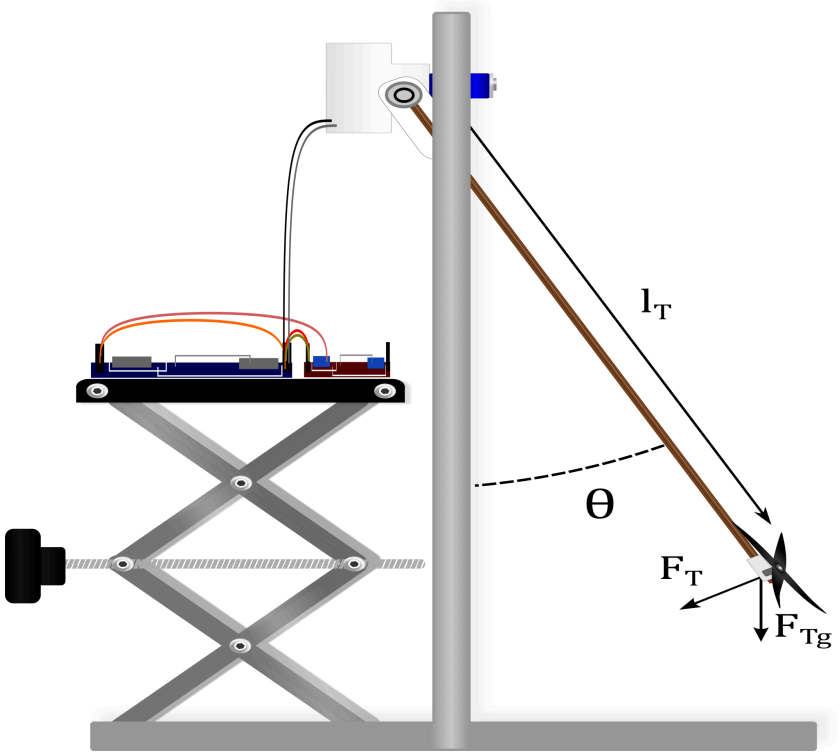
$$M_g = mgl \sin\theta, \tag{2}$$

waarbij m de massa van het bewegende systeem is, g de zwaartekrachtsversnelling en θ de hoek van de arm.

Volgens de tweede wet van Newton voor rotatie wordt de beweging bepaald door de som van deze momenten:

$$J\ddot{\theta} = l_T F_T - mgl \sin\theta, \tag{3}$$

waarbij J het massatraagheidsmoment van het aeropendulum rond het draaipunt is.



Figuur 1: Schematische weergave van het aeropendulum en de relevante krachten.

De stuwkracht van de propeller wordt bepaald door het vermogen dat aan de motor wordt geleverd. Het elektrisch ingangsvermogen is

$$P_{\text{el}} = UI, \tag{4}$$

met U de spanning over de motor en I de stroom door de motor. Slechts een deel van dit vermogen wordt door de motor en propeller omgezet in mechanisch vermogen. Met een rendement η kan dit worden geschreven als

$$P_{\text{motor}} = \eta U I. \tag{5}$$

De stuwkracht kan daarom worden beschouwd als een functie van het geleverde vermogen,

$$F_T = f(P_{\text{motor}}). \tag{6}$$

De precieze vorm van deze relatie hangt af van onder andere de motor en de propeller en hoeft niet lineair te zijn.

Wanneer het aeropendulum uiteindelijk tot stilstand komt, is de hoekversnelling nul. Het moment van de propeller is dan gelijk aan het moment van de zwaartekracht:

$$l_T F_T = mgl \sin\theta. \tag{7}$$

Hieruit volgt

$$\theta = \arcsin\left(\frac{l_T F_T}{mgl}\right). \tag{8}$$

Een groter motorvermogen leidt in het algemeen tot een grotere stuwkracht en daarmee tot een grotere evenwichtshoek. Tijdens het practicum wordt deze relatie experimenteel onderzocht door het motorvermogen te variëren en de resulterende hoek van het aeropendulum te meten.

Componenten

Het aeropendulum bestaat uit verschillende elektronische en mechanische componenten. De Arduino stuurt de motor aan via een H-brug en leest tegelijkertijd de hoek van het pendulum uit met een magnetische hoeksensor. Figuur 3 geeft een overzicht van de gebruikte aansluitingen.

Arduino Uno R4 Minima

Een Arduino is een programmeerbaar microcontrollerbord waarmee elektronische componenten kunnen worden aangestuurd en sensoren kunnen worden uitgelezen. In dit practicum wordt een **Arduino Uno R4 Minima** gebruikt.

De Arduino voert continu een programma uit waarin twee taken centraal staan: het aansturen van de elektromotor en het meten van de hoek van het aeropendulum. Hiervoor beschikt de Arduino onder andere over digitale aansluitingen, analoge ingangen en verschillende communicatieprotocollen.

In deze opstelling worden de digitale pinnen D9, D10 en D11 gebruikt voor de aansturing van de motor. De AS5600-hoeksensor communiceert via de pinnen A4 en A5 met de Arduino.

H-brug

Een elektromotor kan niet rechtstreeks vanuit een digitale uitgang van de Arduino worden gevoed. De motor vraagt aanzienlijk meer stroom dan een uitgang van de microcontroller kan leveren. De Arduino levert daarom alleen het *stuursignaal*; het elektrische vermogen voor de motor wordt geleverd door de externe voeding.

Tussen de Arduino en de motor bevindt zich daarom een **L298N H-brug**. Deze schakeling kan de externe voedingsspanning naar de motor schakelen. De naam H-brug komt van de configuratie van de interne schakelaars, die schematisch de vorm van de letter H heeft.

Door verschillende schakelaars te openen en sluiten kan de stroom door de motor in beide richtingen lopen. Daarmee kan ook de draairichting van de motor worden veranderd. Hiervoor gebruikt de L298N de ingangen IN1 en IN2. In onze opstelling worden deze aangestuurd door D10 en D11 van de Arduino.

Pulse Width Modulation

Naast de draairichting willen we ook kunnen regelen hoeveel vermogen de motor ontvangt. Hiervoor wordt **Pulse Width Modulation (PWM)** gebruikt.

Bij PWM wordt de voedingsspanning niet continu veranderd. In plaats daarvan wordt de spanning snel aan- en uitgeschakeld. De verhouding tussen de tijd dat het signaal aan staat en de totale periodetijd noemen we de *duty cycle* D:

$$D = \frac{t_{\text{aan}}}{T}. \tag{9}$$

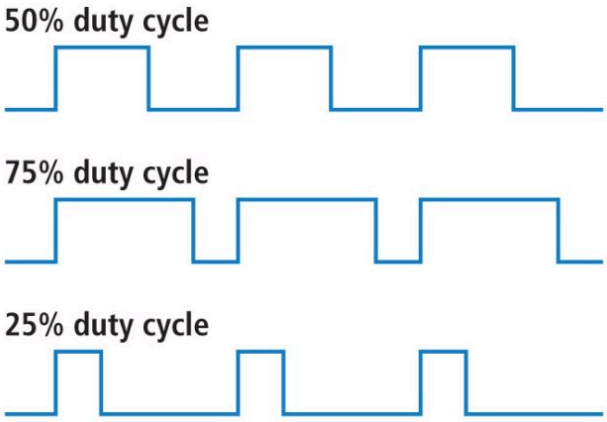
Een duty cycle van 100% betekent dat de motor continu wordt aangestuurd. Bij 50% staat de spanning gedurende de helft van iedere periode aan en bij 0% staat de motor volledig uit.

Voor een ideale PWM-schakeling is de gemiddelde spanning over één periode

$$U_{\text{gem}} = D\,U_{\text{voeding}}. \tag{10}$$

De motor reageert door zijn mechanische en elektrische traagheid niet afzonderlijk op iedere snelle puls. Daardoor kan met de duty cycle het gedrag van de motor worden geregeld.

In onze opstelling genereert pin D9 van de Arduino het PWM-signaal. Dit signaal wordt aangesloten op de **ENA**-ingang van de L298N. De Arduino bepaalt dus wanneer de H-brug de motor inschakelt, terwijl het daadwerkelijke motorvermogen uit de externe voeding komt.



Figuur 2: Voorbeeld van PWM-signalen met verschillende duty cycles. De amplitude blijft gelijk, maar de verhouding tussen de aan- en uittijd verandert.

AS5600-hoeksensor

De hoek van het aeropendulum wordt gemeten met een **AS5600 magnetische hoeksensor**. Op de draaiende as van het aeropendulum wordt een magneet geplaatst. De sensor meet de richting van het magnetische veld en kan daaruit de hoek van de as bepalen. Omdat er geen mechanisch contact nodig is tussen de sensor en de draaiende as, treedt hierbij vrijwel geen extra mechanische weerstand op.

De AS5600 heeft een resolutie van 12 bit. Eén volledige omwenteling wordt daardoor verdeeld in

$$2^{12} = 4096 \tag{11}$$

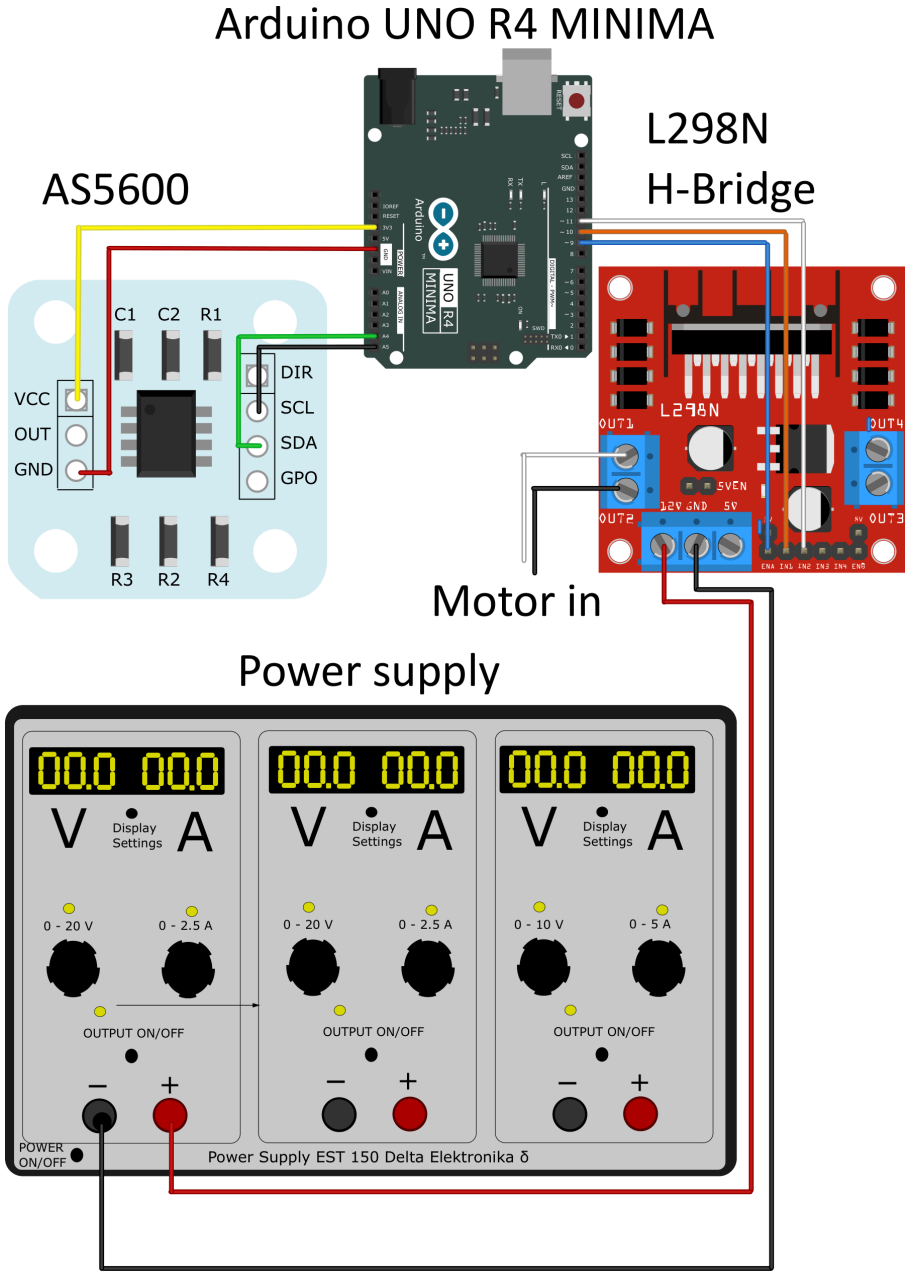
mogelijke waarden. Een ruwe sensorwaarde kan worden omgerekend naar een hoek volgens

$$\theta = \frac{\text{sensorwaarde}}{4096}\,360^\circ. \tag{12}$$

De sensor wordt digitaal uitgelezen via het **I²C-protocol**. Hiervoor zijn twee verbindingen nodig: SDA (*Serial Data*) voor het versturen van gegevens en SCL (*Serial Clock*) voor het kloksignaal. In deze opstelling is SDA aangesloten op A4 en SCL op A5 van de Arduino.

De positie van de magneet ten opzichte van de sensor is belangrijk. Wanneer de magneet te ver van de sensor staat of niet goed boven het midden van de sensor is geplaatst, kan de hoekmeting

onbetrouwbaar worden. Controleer daarom voor iedere meting of de sensor correct functioneert. Wanneer de arm langzaam met de hand wordt bewogen, moet de gemeten hoek vloeiend veranderen. Een constante waarde, grote sprongen of een foutmelding kan betekenen dat de magneet niet correct ten opzichte van de sensor staat.



Figuur 3: Aansluitschema van de Arduino Uno R4 Minima, L298N H-brug, AS5600-hoeksensor, motor en externe voeding.

Voorbereiding

1 Bekijk de geometrie van het aeropendulum. Onder welke voorwaarde geldt $l = l_T$? Leg in woorden uit wat deze aanname fysisch betekent.

2 In de stationaire toestand staat de arm stil en zijn de momenten rond het draaipunt in evenwicht. Laat zien dat dan geldt:

$$l_T F_T = mgl \sin\theta. \tag{13}$$

3 De motor zet niet al het opgenomen elektrische vermogen om in mechanisch vermogen. Neem een rendement van $\eta = 0.70$ aan:

$$P_{\text{mech}} = \eta U I. \tag{14}$$

Bereken bij een voedingsspanning van $U = 4$ V het mechanisch vermogen voor een aantal verschillende motorstromen.

4 Neem aan dat de stuwkracht van de propeller evenredig is met het mechanisch vermogen:

$$F_T = k_P P_{\text{mech}}, \tag{15}$$

waarbij k_P een constante is die de eigenschappen van de motor en propeller beschrijft. Combineer de bovenstaande vergelijkingen en leid een uitdrukking af voor de stationaire hoek θ als functie van de motorstroom I.

5 Kies realistische waarden voor m, l, l_T en k_P. Licht kort toe waarom je deze waarden hebt gekozen. Maak vervolgens in Desmos een grafiek van θ als functie van I.

6 Voorspel op basis van je grafiek wat er gebeurt wanneer de motorstroom wordt verhoogd. Verwacht je dat het eenvoudige model voor alle motorstromen even goed werkt? Leg kort uit.

Meten vanuit de Arduino IDE

Gebruik de CSV-sketch voor dit practicum in `Arduino/Openloop` of `Arduino/PID`. Open voor open-loopmetingen `Aero_pendulum_OL_CSV.ino` en voor PID-metingen `Aero_pendulum_PID_CSV.ino`, ieder in zijn eigen gelijknamige map. Het programma draait volledig op de Arduino. Python wordt pas achteraf gebruikt voor de analyse.

1. Selecteer in de Arduino IDE de Arduino Uno R4 Minima en de juiste poort. Installeer `AS5600` van Rob Tillaart. Alleen de PID-sketch heeft daarnaast de bibliotheek `PID` van Brett Beauregard nodig.
2. Controleer met de motorvoeding uit de bedrading en hoekconversie. Stel met de begeleider `MAX_PWM`, `MIN_ANGLE_DEG` en `MAX_ANGLE_DEG` vast. De meegeleverde waarden zijn voorlopig.
3. Wijzig voor open-loop `OPEN_LOOP_PWM`; wijzig voor PID `setpoint`, `kp`, `ki` en `kd`. De proefduur staat in `MEASUREMENT_MS` en is standaard 30000 ms. Upload met uitgeschakelde motorvoeding.
4. Open Serial Monitor en kies 115200 baud. Schakel de weergave van extra tijdstempels uit: de sketch levert zelf de tijd. Sluit Serial Plotter en andere programma's op dezelfde poort.
5. Verstuur de hoofdletter `C` voor tien seconden sensorcontrole zonder motoraansturing. Controleer de hoek met alleen USB aangesloten. De uitvoer stopt daarna vanzelf. Een ongeldige sensorstatus of werkhoek levert een foutregel en beëindigt de opname.
6. Maak Serial Monitor leeg vóór de echte meting. Schakel de gecontroleerde motorvoeding in terwijl de sketch wacht. Houd afstand van arm en propeller en verstuur `S`. De Arduino start de motor en verstuurt een CSV-kop gevolgd door meetregels. Een eventuele regeleinde-instelling mag blijven staan; CR en LF worden genegeerd.
7. Na de ingestelde proefduur stopt de Arduino de motor en de uitvoer automatisch. Verstuur `X` om eerder te stoppen. Bij een sensorfout of een hoek buiten het ingestelde bereik stopt de motor eveneens. Schakel na afloop de motorvoeding uit.

Het sluiten van Serial Monitor is geen stopcommando. Gebruik `X` of schakel de motorvoeding uit. Houd de fysieke uitschakeling bereikbaar: de softwarematige stop werkt alleen zolang de microcontroller zijn programma blijft uitvoeren.

CSV-bestand bewaren

De Arduino verstuurt CSV-tekst via USB; zonder opslagmodule schrijft hij geen bestand op de computer. Bewaar de meetgegevens als volgt:

1. Wacht totdat de uitvoer is gestopt. Selecteer en kopieer de volledige opname uit Serial Monitor, vanaf de regel die begint met `t_ms,setpoint_deg` tot en met de laatste volledige meetregel.
2. Plak deze tekst in een leeg tekstbestand, bijvoorbeeld in Kladblok. Sla op als `pid_run1.csv` of `ol_pwm60_run1.csv`. Kies bij opslaan zo nodig "Alle bestanden", zodat de naam niet eindigt op `.csv.txt`.
3. Gebruik een nieuwe bestandsnaam per proef. Neem geen meldingen van de IDE, geweigerde startmeldingen of een tweede CSV-kop op in hetzelfde bestand. Controleer bij langere proeven of het begin van de uitvoer nog aanwezig is; verkort de proef als de monitor oude regels niet bewaart.

De belangrijkste kolommen zijn `t_ms` (tijd sinds het startcommando), `angle_deg` (hoek), `pwm` (werkelijke PWM-telwaarde), `setpoint_deg` (leeg bij OL), `running` en `sensor_ok`. Noteer de gekozen PWM- of PID-instellingen en de maximale PWM zelf in het labjournaal. Een ongeldige hoek blijft leeg. Het scheidingsteken is een komma en het decimaalteken een punt. Noteer voedingsinstellingen en kalibratie apart in het labjournaal.

Meten, regelen en CSV-uitvoer gebeuren in dezelfde ronde, nominaal iedere 20 ms (50 Hz). Gebruik de tijdstempels voor de tijdas, met $t = 0$ bij het startcommando. De eerste meetregel volgt enkele milliseconden later. Controleer de werkelijke intervallen door opeenvolgende tijdstempels van elkaar af te trekken. Er is geen nulmeting vóór de motoraansturing; bepaal de beginhoek vooraf met `C`. Verdere instructies staan in `Arduino/README.md`.

Methode

- Gebruik de nieuwe sketch `Aero_pendulum_OL_CSV.ino` in de gelijknamige map onder `Arduino/Openloop`.
- Houd de voedingsspanning en stroomlimiet gelijk tijdens de metingen. Varieer `OPEN_LOOP_PWM` in vooraf afgesproken stappen binnen `MAX_PWM`.
- Schakel de motorvoeding uit voordat je een gewijzigde instelling uploadt. Voer daarna de start- en opslagprocedure hierboven uit.
- Wacht per instelling tot de arm een stationaire positie bereikt, of noteer dat dit binnen de meetduur niet gebeurt.
- Herhaal iedere instelling minimaal drie keer. Registreer per proef een apart CSV-bestand en noteer voedingsspanning en motorstroom voor de vergelijking met het fysische model.

Uitvoering

1. Maak een meettabel met de ingestelde PWM-telwaarde en de gemeten stationaire hoek.
2. Meet de stationaire hoek voor alle ingestelde PWM-waarden.
3. Herhaal iedere meting minimaal drie keer.
4. Noteer eventuele bijzonderheden, zoals een dode zone, oscillaties of verzadiging van de motor.

Verwerking

1. Laad de meetgegevens in met Python.
2. Controleer de CSV-kop: het scheidingsteken is een komma en het decimaalteken een punt. Analyseer alleen regels met `running=1` en `sensor_ok=1`.
3. Print de kolomnamen met `df.keys()`.
4. Bereken voor iedere PWM-instelling de gemiddelde stationaire hoek en de standaarddeviatie.
5. Maak een grafiek van de stationaire hoek als functie van de ingestelde PWM-telwaarde, inclusief foutbalken.
6. Vergelijk de gemeten relatie met de theoretische voorspelling uit de voorbereiding en bespreek eventuele afwijkingen.

Importeer data

```
import pandas as pd
df = pd.read_csv('ol_pwm60_run1.csv')
df = df[(df['running'] == 1) & (df['sensor_ok'] == 1)].copy()
if df.empty:
    raise ValueError('Geen geldige actieve meetpunten')
df['t_s'] = df['t_ms'] / 1000

print(df.keys())
```

Verwerking

1. Plot de stationaire hoek als functie van de ingestelde PWM-telwaarde. Voeg foutbalken toe indien meerdere metingen zijn uitgevoerd.
2. Vergelijk de theoretische hoek als functie van stroom met de apart gemeten motorstroom en stationaire hoek. Een PWM-telwaarde is geen gemeten spanning of stroom; zet deze niet rechtstreeks op de stroomas van het model.
3. Bepaal voor welke uitgang de propeller de arm voor het eerst in beweging brengt (de dode zone).

4 Onderzoek of de relatie tussen de Arduino-uitgang en de stationaire hoek lineair is. Licht je antwoord toe.
5 Bespreek mogelijke oorzaken van afwijkingen tussen het theoretische model en de metingen, zoals wrijving, niet-lineair motorgedrag, luchtweerstand en meetonzekerheid.

Afronding

1 Laat de meetresultaten controleren door de practicumbegeleider.
2 Voeg alle grafieken toe aan je digitale labjournaal en geef iedere figuur een duidelijk onderschrift.
3 Vul de sectie **Resultaten en Discussie** aan. Beschrijf hierin de relatie tussen de Arduino-uitgang en de stationaire hoek en vergelijk de experimentele resultaten met het theoretische model.
4 Reflecteer op de beperkingen van een open-loop systeem en beschrijf waarom terugkoppeling in veel technische toepassingen gewenst is.